

11 Функциональ программчлалын парадигм

Энэ бүлэгт бид тооцоолол нь төлөвийг өөрчилж биш харин илэрхийллийг дахин боловсруулах (rewriting) замаар явагддаг функциональ програмчлалын парадигмын талаар авч үзэх болно. Энэ парадигмын хэлүүд нь "дагнасан (pure)" хэлбэрээрээ санах ойн тухай ойлголтыг ашигладаггүй (тиймээс ямар ч гадаад нөлөө (side effect) байхгүй). Орчныг тохируулсны дараа илэрхийлэл нь үргэлж ижил утгыг илэрхийлдэг.

Бид эхний хэсгүүдэд дагнасан парадигмын талаар ярилцаж, түүний суурь ойлголт талаас нь тайлбарлах болно. Бодит функциональ програмчлалын хэлүүд эдгээр "дагнасан" хэсгүүдийг бусад олон механизмтай нэгтгэдэг; Бид тэдгээрийн заримыг 11.3-р хэсэгт авч үзэх болно. Мөн ихэнх орчин үеийн императив хэл нь функциональ програмчлалын хэв маягт зориулсан бүтцийг агуулсан байдаг. Бид сүүлийн хэдэн арваны турш функциональ програмчлалын парадигмыг ихээхэн сонирхох болсон шалтгаануудын талаар ярилцаж, бүлгийн эхний хэсгийг дуусгана.

Бид өндөр эрэмжийн функциональ хэлэнд зориулсан хийсвэр машин болох SECD машиныг авч үзэх бөгөөд энэ нь олон бодит хэрэгжилтийн прототип болж байдаг.

Бүлэг нь бүхий функционал программчлалын хэлний тооцоололд нь зориулагдсан формал систем бөгөөд Algol болон Lisp-ийн үеэс эхлэн программчлалын хэлний дизайнд хэвшсэн загвар нь болсон λ -calculus-ын тухай товч танилцуулгыг агуулсан илүү онолын агуулгаар төгсөнө.

11.1 Төлөвгүйгээр тооцоолох

Уламжлалт хэл нь *төлөв* шилжилт дээрх тооцооллын загварт суурилдаг. Энэ загварын гол цөм нь *засварлагдах боломжтой хувьсагчийн (modifiable variable)* тухай ойлголт, өөрөөр хэлбэл, тооцооллын явцад өөр өөр утга олгож болдог бөгөөд тухайн орчиндоо үргэлж нэг ижил холбоостой байдаг нэр ашигласан контейнер. Уламжлалт хэлний гол бүтээвэр нь хувьсагчийн нэр болон харгалзах байрлал хоорондын холбоосыг өөрчлөхгүйгээр хувьсагчийн утгыг засварладаг *утга олголт* байна. 6.2.2-р хэсгийн томъёоллоор энэ нь хувьсагчийг зарлах үед тохируулсан l-утгыг биш тухайн r-утгыг засварладаг. Энэхүү уламжлалт "имперав" тооцооллын загвар нь анхны электрон хадгалалттай программ компьютерын нэгийг зохион бүтээлцэгч Унгар-Америкийн математикчийн нэрээр хожим 1640-өөд оны үеээс нэрлэгдэх болсон фон Нейман машины загварын дагуу байршилд хадгалагдсан утгыг засварлах замаар тооцоолол нь явагддаг байх физик машины хийсвэр тал юм.

Цэвэр функциональ хэлэнд төлөв эсвэл засварлагдах боломжтой хувьсагч байдаггүй. Зарчмын хувьд тооцоолол нь зөвхөн орчинд тохиолдох өөрчлөлтүүдээр, илэрхийлэлийг *дахин бичилтээр (rewriting)* явагдах бөгөөд санах ойн тухай ойлголтыг агуулдаггүй. Хэрэв засварлагдсах боломжтой хувьсагч байхгүй бол утга олголт хэрэгцээгүй болно. Тооцоолол нь функцийн (дээд эрэмбийн, 7.2-р хэсгийг үзнэ үү) зохицуулалтаар үндсэндээ бий болж байдаг тухайн орчны нөхцөл байдлаар бүхэлдээ илэрхийлэгддэг.

Утга олголтгүйгээр давталтын ач холбогдол буурна. Давталт нь тусгай хувьсагчийн утга хязгаарыг хангах хүртэл төлөвийг давтан өөрчилдөг. Давталтын болон рекурсив бүтцүүд нь төгсгөлгүй үргэлжилсэн (болон магадгүй салаалсан) тооцоолол хийх боломжийг олгодог эквивалент хоёр механизм гэдгийг уншигчид өмнө нь мэддэг болсон (6-р бүлгийг үзнэ үү). Төлөвгүй (stateless) тооцооллын загварын хувьд (хязгаарлалтгүй) давталт байхгүй болж, рекурсив үлддэг бөгөөд

дараалал хяналтад зорулсан суурь зохион байгуулалт болдог. Төгсдөг давталт нь өндөр эрэмбийн функцээр илэрхийлэгддэг, 11.3.4-р хэсгийг үзнэ үү.

Төлөвт суурилсан энгийн *императив парадигмаас* ялгаатай нь өндөр эрэмбийн функц, рекурс нь төлөвгүй тооцооллын загварын үндсэн бүрэлдэхүүн хэсэг болдог бөгөөд үүнийг *функциональ програмчлалын парадигм* гэж нэрлэдэг.

Энэ бол императивтайгаа адил хуучин парадигм. 1930-аад оноос хойш Тьюринг машины зэрэгцэн, (3.4-р хэсэг дэх хайрцгийг үзнэ үү) бидний товч тайлбарласантай нэгэн ижил үзэл баримтлалд суурилсан тооцоолж болох функцэд зориулсан хийсвэр загвар болох λ -тооцоолол бий болсон. Lisp бол λ -тооцооллоос сэдэл авсан анхны програмчлалын хэл байсан ба олон сүүлийн жилүүдэд хэл нь үүнийг баримталсан (Scheme, ML нь өөр өөр хэлбэрээ Миранда, Хаскелл, Эрланг гэх мэт). Эдгээрээс зөвхөн Миранда, Хаскелл нар л “дагнасан функциональ”; бусад хэлүүд нь мөн императив бүрдэл хэсэгтэй байдаг бөгөөд тэдгээрт дэд үүрэг оноодог.

Бид эхлээд дагнасан функциональ парадигмыг танилцуулна. Шинжилгээг илүү тодорхой болгохын тулд бид дундын псевдокодын оронд тодорхой хэлийг ашиглана. Үүний тулд ML-ийг сонгосон бөгөөд учир нь дизайны үялдаа, ойлгомжтой байдлаараа сургалтын (дидактик) танилцуулгад хамгийн тохиромжтой байдаг. Энэ хэлийг танилцуулах нь бидний зорилго биш (үншигч ном зүйд хандах ёстой), гэхдээ бусад функциональ хэлний зарим ерөнхий асуултуудыг хэлэлцэхийн тулд уг синтаксийг зөвхөн багаж хэрэгслээр ашиглана.

11.1.1 Илэрхийлэл ба функц

Ердийн математик практикт бид функцийг хэзээ тодорхойлж, хэзээ түүнийг утга дээр хэрэглэх нь заримдаа тодорхойгүй байдаг. Ийм илэрхийлэлтэй тулгарах нь ховор биш юм:

$f(x) = x^2$ нь x -ийг квадраттай нь холбосон функц байг. Хэрэв $x = 2$ байвал $f(x) = 4$ болно.

Синтакс илэрхийлэл $f(x)$ нь тэс өөр хоёр зүйлийг илэрхийлэхэд хэрэглэгддэг: эхний удаад энэ нь тодорхой функцийг f нэрийг танилцуулахад үйлчилдэг; хоёр дахь удаадаа f функцийг тусгай утгад хэрэглэсэн үр дүнг илэрхийлэхэд үйлчилнэ. Математик практикт контекст нь хоёр хэрэглээний алийг нь ашиглахыг ялгахад тусалдаг учраас энэ хоёрдмол утга нь бүрэн гэм хоргүй. Функциудыг дүрсэлж хэрэглэдэг зохиомол хэлэнд (програмчлалын хэл гэх мэт) тийм биш. Энд хоёр тохиолдлыг ялгах нь тохиромжтой.

Математикч $f(x)$ функцийг тодорхойлж байна гэж батлах үед бодит байдал дээр тэд f функцийг нэг формал параметр болох x -ээр тодорхойлж байгаа бөгөөд энэ нь f нь түүний аргументад хамаарах хувиргалтыг заадаг. Функцийн нэр болон “их бие” хэл шинжлэлийн хувьд ялгахын тулд ML синтаксийг дагуу бид дараах байдлаар бичиж болно:

```
val f = fn x => x*x;
```

val гэсэн нөөц (reserved) үг нь зарлалтыг танилцуулна. Энэхүү зарлалтыг нэр болон утга хоорондын шинэ холбоогоор орчныг өргөтгөхөд ашигладаг. Манай тохиолдолд f нэр нь x -г $x * x$ болгон хувиргахтай холбоотой. Бүх функциональ хэлэнд функц нь *илэрхийлэгдэх боломжтой* утга байдаг; өөрөөр хэлбэл тэдгээр нь комплекс илэрхийллийн үнэлгээний үр дүн байж болно. Манай тохиолдолд $=$ -ийн баруун талд байгаа илэрхийлэл буюу fn -ээр танилцуулсан байгаа нь функцийг тэмдэглэдэг илэрхийлэл юм.

Аргумент дээрх функцийн хэрэглээний хувьд f функцийг аргумент 2-т хэрэглэхэд үүсэх илэрхийлэлд зориулж $f(2)$ эсвэл $(f\ 2)$ эсвэл $f\ 2$ гэж бичдэг уламжлалт тэмдэглэгээг хэвээр үлдээнэ. Мөн `val`-г шинэ нэр танилцуулахдаа ашиглана.

```
val four = f 2;
```

Функцийг тэмдэглэж буй илэрхийлэлд тусгай синтакс оруулах нь чухал үр дагавартай. Функциэд заавал нэр өгөхгүйгээр бичих (болон ажиллуулах) боломжтой. Жишээлбэл, уг илэрхийлэл:

```
(fn y => y+1) (6);
```

нь $fn\ y \Rightarrow y+1$ (нэргүй) функцийн аргумент 6 дээрх хэрэглээнээс 7 гарна утгатай байна. Тэмдэглэгээг илүү онцлохгүйн тулд бид програмыг шууд зэрэгцүүлэн (энэ нь хаалтгүй) тэмдэглэж болох ба энэ нь зүүн талд байгаа холбоосууд гэж бид үзэж байна (prefix тэмдэглэгээ). Хэрэв g нь функцийн нэр бол,

```
g a1 a2 ... ak
```

гэсэн утгатай болно:

```
(...((g a1) a2)... ak).
```

Нэг функциональ илэрхийлэл нөгөө дотор гарч ирэхэд хориглох зүйл байхгүй

```
val add = fn x => (fn y => y+x);
```

`add` утга бол y аргумент өгөхөд $x+y$ -г буцаадаг нэргүй функцийг, x аргумент өгсөн үед буцаадаг функц.

```
val three = add 1 2;  
val addtwo = add 2;  
val five = addtwo 3;
```

Ялангуяа `addtwo` нь өөр илэрхийллийн үнэлгээний үр дүнд олж авсан функц гэдгийг анхаарна уу.

`val` болон `fn` хэрэглэгдэг тэмдэглэгээ нь чухал боловч бага зэрэг нуршуу байна. ML нь програмчлалын хэл дээр функц тодорхойлох ердийн аргатай төстэй илүү хураангуй тэмдэглэгээ ашиглах боломжийг олгодог.

Бидний тодорхойлсон анхны f функцийг (аргументынхаа квадратыг тооцоолох функц) мөн дараах байдлаар тодорхойлж болно:

```
fun f x = x*x;
```

Ерөнхийдөө, уг хэлбэрийн тодорхойлолт:

```
fun F x1 x2 ... xn = body;
```

Энэ нь ердөө syntactic sugar (өөрөөр хэлбэл програмчлалын хэлний хувьд, ердөө сайхан товчлол).

```
val F = fn x1 => (fn x2 => ... (fn xn => body )...);
```

Функц удирддаг функцийн сүүлийн жишээ болгон формал параметр хэлбэрээр дараах тодорхойлолтыг авч үзье.

```
fun comp f g x = f(g(x));
```

Энэ нь эхний хоёр аргументаас бүрдсэн функцийг буцаана (энэ нь бас функцууд юм).

Эцэст нь функциональ програм бүр рекурсив функцийг тодорхойлох боломжийг олгодог. Бидэнд *нөхцөлт илэрхийлэл* байгаа гэж үзвэл (энэ нь ML-д бидний мэдэх if then else синтаксаар бичигдсэн байдаг) бид энгийнц факториалыг дараах байдлаар тодорхойлж болно.

```
fun fatt n = if n=0 then 1 else n*fatt(n-1);
```

11.1.2 Бууралт хэрэгслэн тооцоолол

Хэрэв бид арифметик функц (бид үүнийг ердийн семантикаар урьдчилан тодорхойлсон гэж үзэж болно) болон нөхцөлт илэрхийллийг хасвал ойлголтын түвшинд комплекс илэрхийллийг утга (үнэлгээ) болгон хувиргах процедурыг дахин бичих процесс гэж тодорхойлж болно. Үүнийг бид процесс *бууралт (reduction)* гэж нэрлэдэг. Комплекс илэрхийлэлд "аргументад хэрэглэгдэх функц" хэлбэрийн дэд илэрхийлэл нь формал параметрт байгаа функцийн их биеээр солигдож, энэ нь өөрийн ээлжин дээр бодит параметрээр солигддог.¹ Бид энэхүү тооцооллын загварыг ашиглан энгийн илэрхийлэлийг тооцоолж болно (бид бууралтын алхмыг заахдаа → ашигладаг).

```
fun K x y = x;
fun S p q r = p r (q r);
val a = ...;

S K K a → (fn p => (fn q => (fn r => p r (q r)))) K K a
        → (fn q => (fn r => K r (q r))) K a
        → (fn r => K r (K r)) a
        → K a (K a)
        → (fn x => (fn y => x)) a (K a)
        → (fn y => a) (K a)
        → a
```

Figure 11.1 Зарим тодорхойлолт болон дахин бичих аргыг ашиглан тооцоолох

```
fact 3 → (fn n => if n=0 then 1 else n*fact(n-1)) 3
        → if 3=0 then 1 else 3*fact(3-1)
        → 3*fact(3-1)
        → 3*fact(2)
        → 3*((fn n => if n=0 then 1 else n*fact(n-1)) 2)
        → 3*( if 2=0 then 1 else 2*fact(2-1))
        → 3*(2*fact (2-1))
        → 3*(2*fact(1))
        → 3*(2*((fn n => if n=0 then 1 else n*fact(n-1)) 1))
        → 3*(2*( if 1=0 then 1 else 1*fact(1-1))
        → 3*(2*(1* fact(0))
        → 3*(2*(1*((fn n => if n=0 then 1 else n*fact(n-1)) 0)))
        → 3*(2*(1*( if 0=0 then 1 else n*fact(n-1))))
        → 3*(2*(1*1))
        → 6
```

¹ Уншигч энэ тайлбарт бидний өмнө нь нэрээр параметрийн дамжуулах хүрээнд хэлэлцсэн "хуулбарлах дүрэм"-ийг таньсан байх нь гарцаагүй. Одоохондоо бид энэ процест яг ямар семантик хуваарилах талаар тодорхойгүй байх болно—11.2-р бүлэгт бүгдийг зориулах болно.

Арифметик тооцоолол болон нөхцөлт илэрхийлэлгүй үед тооцооллын бусад бүх үйлдлүүд тэмдэгт мөрийн симболик удирдлагаар хэрхэн явагддагийг анхаарна уу: хувьсагч байхгүй, биелэлтийн стек дээрх утгыг шинэчлэхгүй. Зураг 11.1 нь дан симболик удирдлагын өөр нэг жишээг агуулж байна. (Энэ нь ялгах функцийг бичихэд төвөгтэй арга юм!) (Зургийг шинжлэхдээ `fun` гэдэг нь `val ... fn` гэсэн үгийн товчлол гэдгийг санаарай.)

Эцэст нь хэлэхэд, уншигч

```
fun r x = r(r(x));
```

гэсэн тодорхойлолтыг өгвөл `r`-ийн үнэлгээг агуулсан тооцоолол бүр төгсгөлгүй дахин бичилт болно гэдэгт итгүүлэхэд хүндрэл гарахгүй. Ийм тохиолдолд тооцоолол нь *сарниж (diverges)*, үр дүн нь тодорхойгүй байна гэж хэлнэ.

11.1.3 The Fundamental Ingredients

Эдгээр эхний дэд хэсгүүдэд бид функциональ дагнасан парадигмын үндсэн бүрдэл хэсгүүдийг танилцуулсан. Үүнийг бид тодорхой болгож, үндсэн ойлголтуудыг дараах байдлаар нэгтгэн дүгнэж болно.

Синтаксийн үүднээс авч үзвэл цэвэр функциональ хэл нь командгүй (гадаад нөлөөллийг ашиглан өөрчлөх төлөв байхгүй), зөвхөн илэрхийлэл юм. Өгөгдлийн (бүхэл тоо, `boolean`, тэмдэгт гэх мэт) болон нөхцөлт илэрхийллийн боломжит утгууд, анхдагч үйлдлүүдээс гадна илэрхийллийг тодорхойлох үндсэн хоёр бүтэц байдаг:

- Аливаа илэрхийлэл `exp` болон `identifier` `x`-ийн хувьд *хийсвэрлэл* нь `x` формал параметрийг `exp` болгон хувиргах функцийг илэрхийлэх `fn x => exp` илэрхийллийг бүтээх боломжийг олгодог (`ex` илэрхийлэл нь `x`-тэй холбогдсон тодорхой утгаас "хийсвэрлэсэн").
- Бид `f_exp` `a_exp` гэж бичдэг өөр илэрхийлэл болох `a_exp`-д `f_exp` илэрхийллийн хэрэглээ бөгөөд энэ нь `f_exp` функцийг `a_exp` аргументад хэрэглэхийг илэрхийлдэг.

Бусад функцэд аргумент болгон дамжуулах эсвэл бусад (өндөр эрэмбийн) функцийн үр дүн болгон буцаахад функцүүдэд хязгаарлалт байхгүй. Иймээс программ ба өгөгдөл хоёрын хооронд хангалттай нэгэн *төрлийн байдал (homogeneity)* оршдог.

Семантик үүднээс авч үзвэл, программ нь хэд хэдэн утгын тодорхойлолтоос бүрддэг бөгөөд тэдгээр нь тус бүр нь тухайн орчинд шинэ холбоосыг бий болгодог бөгөөд дурын комплекс илэрхийллийн үнэлгээг шаарддаг. Өндөр эрэмжийн функц, рекурсив функц боломж нь энэхүү тодорхойлолтын механизмыг уян хатан, хүчтэй болгодог.

Анхны байдлаар ойролцоогоор дүгнэхэд тооцооллын (*үнэлгээний*) семантик нь өнөөг танилцуулсан зүйлээс өөр хэл шинжлэлийн өнцөгтэй холбогдохгүй. Энэ нь хоёр үндсэн үйлдлийг давтан хэрэглэж илэрхийллийг хялбаршуулсаар утга шууд заасан энгийн хэлбэртэй болгодог тэмдэгт мөрийн хялбар симболик дахин бичилт (бууралт)-ийг ашигладаг. Эдгээр үйлдлүүдийн эхнийх нь тухайн орчин дахь энгийн хайлт юм. Ялагч нь орчинд холбогдсон гэж тогтоогдох үед тодорхойлолтоор нь солино. Жишээлбэл, 11.1-р зурагт бид энэ үйлдлийг эхний болон тав дахь бууралт дээр ашигласан. Үүнээс хойш энэ алхамыг дахиад ил авч үзэхгүй. Бид нэрийг түүнтэй холбоотой утгын энгийн товчлол гэж үзэх болно.

Илүү сонирхолтой хоёр дахь үйлдэл нь аргументад хэрэглэгдэх функциональ илэрхийлэлийг авч үздэг ба хуулах (сору) дүрмийн хувилбарыг ашигладаг (энэ агуулганд үүнийг β -дүрэм гэдэг).

Definition 11.1

Redex Redex (энэ нь *бууруулж болох* илэрхийлэл) нь $((fn\ x \Rightarrow body)\ arg)$ хэлбэрийн хэрэглээ юм.

Reductum Redex-ийн *reductum* $((fn\ x \Rightarrow body)\ arg)$ гэдэг нь *body* дахь формал параметр болох *x*-ийн тохиолодол бүрийг *arg*-ийн хуулбараар солин гаргаж авсан илэрхийлэл юм (хувьсагч олж авдаг байхаас зайлсхийх,² 7.2.2-ыг үзнэ үү).

β -дүрэм Redex нь дэд илэрхийлэл болж гарч ирэх *exp* илэрхийлэл нь *exp1* (тэмдэглэгээ: $exp \rightarrow exp1$) болж бүүрдаг (эсвэл дахин бичих, хялбаршуулах) бөгөөд *exp1*-ийг *reductum*-аар солин *exp*-ээс гарган авдаг.

Хэрэгжилтийн үүднээс авч үзвэл, функциональ хэлэнд зориулсан хийсвэр машин бүр хаягдал цуглуулах өргөтгөсөн механизмыг ашигладаг, учир нь функцийг бусад функцийн утга болгон буцаах тохиолдолд локал орчин хязгааргүй хугацаанд хадгалагдах ёстойг бид мэднэ (7.2.2-ыг үзнэ үү).

Хэрэв эдгээр нь функциональ хэлний үндсэн ойлголтууд юм бол илүү анхааралтай уншихад эдгээр цөөн хэдэн мөр нь шийдвэрлэхээсээ илүү олон асуудал үүсгэдэг. Бид дараагийн хэсэгт эдгээр ойлголтуудад илүү гүнзгий дүн шинжилгээ хийх болно.

11.2 Үнэлгээ

Сүүлийн хэсэгт дор хаяж хоёр асуудал нээлттэй байна:

- Бууралтын төгсгөлийн нөхцөл нь юу вэ ("утгыг шууд илэрхийлдэг энгийн хэлбэр" гэж юу вэ?)
- β -дүрэмд ямар нарийвчилсан семантик өгөх ёстой вэ, зөвхөн боломжит хувьсагчийг олж авах тухай биш (бид үүнийг параметр дамжуулалтын хүрээнд өмнө тайлбарласан), харин илүү чухал нь дахин бичих явцад дагаж мөрдөх шаардлагаар нэг илэрхийлэлд нэгээс илүү *redex* байх үү?

11.2.1 Утга

Утга нь дахин бичих боломжгүй илэрхийлэл юм. Функциональ хэлэнд хоёр төрлийн утга байдаг: анхдагч төрлийн утга ба функц. Энгийн төрлийн утгын талаар ярих зүйл бага. Хэл нь зарим анхдагч төрлөөр (бүхэл тоо, *boolean*, тэмдэгт гэх мэт) хангаж өгсөн бол уг төрөл бүр нь үнэлгээг хүлээн зөвшөөрдөггүй утгын олонлог байх нь ойлгомжтой (жишээлбэл, бүхэл тоон тогтмол, *boolean*, тэмдэгт). Өмнөх дэд хэсгийн жишээн дээр бид бүхэл тоон төрлийн үндсэн утганд хүрсэн үед *fact*-ын үнэлгээг зогсоосон: 3, 2, 1.

Функциональ утгууд нь илүү сонирхолтой байдаг. Дараахь тодорхойлолтыг авч үзье:

```
val G = fn x => ((fn y => y+1) 2);
```

Тодорхойлолт нь тэнцүүгийн баруун талд байгаа илэрхийлэлийн үнэлгээ, дараа нь үүссэн утгыг $=$ -ийн зүүн талд байгаа нэрээр холбохыг шаарддаг гэж бид хэлсэн. Гэхдээ энэ тохиолдолд *G*-тэй ямар

² Функциональ програмчлалын хүрээнд утгатай холбоотой танигчийг ихэвчлэн "хувьсагч" гэж нэрлэдэг. Уншигч одоохондоо цэвэр функциональ програмчлалын хэл дээр засварлагдах боломжтой хувьсагч байхгүй тул төөрөгдөлгүйгээр энэхүү уламжлалт хэрэглээг үргэлжлүүлэх хангалттай түршлагатай болсон.

утгатай холбоотой байх нь тодорхойгүй байна. Бидэнд: G-ийн их биед агуулагдах редексийг үнэлж дахин бичсэн:

```
fn x => 3
```

байна уу, эсвэл G-ийн их биед ямар ч үнэлгээ хийгээгүй:

```
fn x => ((fn y => y+1) 2)
```

байна уу? Эхний тохиолдол нь бидний сүүлийн дэд хэсгийн төгсгөлд өгсөн формал бус семантикийг илүү харгалзан үздэг тохиолдол юм. Харин хоёр дахь нь императив хэл дээрх функцийг тодорхойлолтын ердийн агуулгад илүү ойр буюу функцийг их биеийг дуудсан үед л үнэлэгддэг.

Нийтлэг хэрэглэгддэг функциональ хэлүүд бүгд эдгээр сонголтуудын сүүлийнхийг ашигладаг - хийсвэрлэлийн "доор" үнэлгээ байдаггүй. Дараах хэлбэрийн илэрхийлэл бүр нь утга

```
fn x => exp
```

тиймээс функцийг аргументад хэрэглэх хүртэл exp-д агуулагдаж болох редексийг хэзээ ч дахин бичихгүй.

11.2.2 Capture-Free Substitution

Capture-Free орлуулгыг хэрэгжүүлэхийн тулд бид хаалт ашиглаж болохыг 7-р бүлэгт харсан. Энэ нь үнэн хэрэгтээ хийсвэр машины функциональ хэлэнд ашигладаг механизм юм (11.5-р хэсгийг үзнэ үү). Одоогийн байдлаар бидний өгч буй энгийн тайлбарын хувьд синтакс дүрэм хангалттай. Илэрхийлэл бүрт ижил нэртэй хоёр формал параметр хэзээ ч байдаггүй бөгөөд формал параметр биш байж болох хувьсагчдын нэрс бүгд формал параметруудээс ялгаатай байдаг.

Энэхүү хэвшлийг ашигласнаар бидний авч үзэж буй энгийн жишээнүүдэд хувьсагч барилт (variable capture) хэзээ ч байхгүй болно.³

11.2.3 Үнэлгээний стратеги

6-р бүлэгт илэрхийллийн талаар ярилцаж байхдаа хэл бүр илэрхийллийг үнэлэх тодорхой стратегийг (энэ нь тогтмол эрэмбэтэй) хэрхэн тохируулах ёстойг олж харсан. Функциональ хэлэнд өндөр эрэмбийн функц байдаг нь энэ асуултыг илүү суурь болгодог. Асуудлыг тодруулахын тулд 11.2-р зураг дээрх тодорхойлолтуудыг авч үзье. Аль утгыг v-тэй холбож, яаж тодорхойлох вэ?

```
fun K x y = x;  
fun r z = r(r(z));  
fun D u = if u=0 then 1 else u;  
fun succ v = v+1;  
  
val v = K (D (succ 0)) (r 2);
```

Figure 11.2 Нэгээс олон редекс бүхий илэрхийлэл

³ Илүү ерөнхий тохиолдолд, хэрэв бид хаалтын тухай ойлголтыг оруулахыг хүсэхгүй байгаа бол дахин бичих аргыг ашиглан тооцооллыг зөв тайлбарлахын тулд 11.6-д формалчилж авч үзсэн хязгаарлагдмал болон чөлөөт хувьсагч, орлуулалт гэсэн ойлголтуудыг сайтар тодорхойлох шаардлагатай.

β -дүрэм нь дангаараа тийм ч их ашиггүй, учир нь v -ийн тодорхойлолтын баруун талд 4 редекс байна (тодорхойлолтын дагуу нэрсийг тэдгээртэй холбоотой утгуудаар өргөжүүлсний дараа):

$K(D(succ\ 0))$

$D(succ\ 0)$

$succ\ 0$

$r\ 2$

Тэдний аль нь хамгийн түрүүнд буурсан байх вэ? Түгээмэл хэрэглэгддэг хэл бүр нь хамгийн зүүн талын стратегийг ашигладаг бөгөөд энэ нь хамгийн зүүн талын төгсгөл дэх редексээс эхлээд хуучирсан бууруулдаг. Гэхдээ үүнийг зааж өгсөн байсан ч

$K(D(succ\ 0))$

$D(succ\ 0)$

$succ\ 0$

эдгээрийн аль нь хамгийн зүүн талд байгаа нь тодорхойгүй, учир нь эдгээр гурван редекс нь бие биендээ давхардсан байна. Хамгийн зүүн талын үнэлгээний эрэмбийг өгснөөр дараах гурван үндсэн өөр стратеги байдаг.

Утгаар үнэлэх

Утгаар үнэлэхэд (үүнийг мөн хэрэглэх эрэмб, эсвэл *eager*, эсвэл *хамгийн дотоод үнэлгээ* гэж нэрлэдэг) аргументийн хэсгийг бүрдүүлдэг илэрхийлэл нь утга болсон тохиолдолд л редексийг үнэлэгддэг. Илүү нарийвчлалтай, хэрэглэгдэх эрэмбийн хамгийн зүүн талын үнэлгээ дараах байдлаар ажиллана:

Scan the expression to be evaluated from the left, choosing the first application encountered. Let it be $(f_exp\ a_exp)$.

1. Эхний тааралдсан ашиглалтыг сонгон зүүн талаас үнэлэх илэрхийллийг сканнердах. $(f_exp\ a_exp)$ байг.
2. $(\lambda x. \dots)$ хэлбэрийн утга руу буурсан байх хүртэл эхлээд f_exp -г үнэлнэ (энэ методыг рекурсивээр ашиглана).
3. Дараа нь энэ ашиглалтын аргументын хэсэг болох a_exp -ийг үнэлж, val гэсэн утга болгон бууруулна.
4. β -дүрмийг ашиглан редексийг $((\lambda x. \dots)val)$ бууруулж (1)-ээс эхлэн давтана.

Зураг 11.2-ыг авч үзвэл (1) тохиолдолд эхлээд $K(D(succ\ 0))$ ашиглалтыг сонгоно. Одоо (1), (2) ба (3) -ын зарим энгийн ашиглалт нь K , D болон $succ$ -г утгууд болсон гэдгийг харуулах болно (нэр нь түүнтэй холбоотой илэрхийллийн товчлол гэдгийг санаарай). Дараа нь буурах эхний редекс нь $succ\ 0$ (өөрөөр хэлбэл $((\lambda v. v + 1)\ 0)$) бөгөөд үүнийг бүрэн үнэлж, 1-ийг гаргана. Дараа нь 1 утгыг өгөхийн тулд $redex\ (D\ 1)$ (энэ нь $((\lambda u. \text{хэрэв } u=0 \text{ бол } 1 \text{ өөр } u)\ 1)$) буурна. Дараа нь $(K\ 1)$ үнэлэмжийг гаргаж үнэлнэ $(fun\ y \Rightarrow 1)$.

$(fun\ y \Rightarrow 1)\ (r\ 2)$

Энэ програмын функциональ хэсэг нь утга болсон тул стратеги нь аргумент ($r\ 2$) үнэлэхийг зааж өгдөг. β -дүрмийн хэрэглээ нь $r\ (r\ 2)$ -д хүргэдэг; дараа нь бид $r\ (r\ (r\ 2))$ гэх мэт ялгаатай тооцоололд шилжинэ.

Тооцоолол нь сарниж байгаа тул v -тэй ямар ч утга холбогдохгүй.

Нэрээр үнэлэх

Нэрээр нь үнэлэх стратегид (энэ нь *хэвийн эрэмбэ* эсвэл *хамгийн гадаад үнэлгээ* гэж нэрлэгддэг) аргументийн хэсгийг эхлээд бууруулахгүйгээр редекс буурдаг. Илүү нарийвчлалтайгаар, хэвийн эрэмбээр хамгийн зүүн талын үнэлгээг доор үзүүлэв.

1. Эхний тааралдсан ашиглалтыг сонгон зүүн талаас үнэлэх илэрхийллийг сканнердах. ($f_exp\ a_exp$) байг.
2. $(fn\ x\ =>\ ...)$ хэлбэрийн утга руу буурсан байх хүртэл эхлээд f_exp -г үнэлнэ (энэ методыг рекурсивээр ашиглана).
3. β -дүрмийг ашиглан редексийг $((fn\ x\ =>\ ...)val)$ бууруулж (1)-ээс эхлэн давтана.

Зураг 11.2-ыг авч үзвэл, хамгийн түрүүнд бууруулж буй редекс нь:

$K\ (D\ (succ\ 0))$

Үүнийг дахин бичсэн байна:

$fn\ y\ =>\ D\ (succ\ 0)$

Энэ нь функциональ утга юм. Одоо бүтэн илэрхийлэл нь дараах хэлбэртэй байна:

$(fn\ y\ =>\ D\ (succ\ 0))\ (r\ 2)$

энэд стратеги нь хамгийн гаднах редекс-г бууруулахаар заасан тул бид дараахь зүйлийг олж авна.

$D\ (succ\ 0)$

Одоо энэ илэрхийлэлийг бууруулснаар бид дараахь зүйлийг олж авна.

$if\ (succ\ 0)=0\ then\ 1\ else\ (succ\ 0)$

дараа нь:

$if\ 1=0\ then\ 1\ else\ (succ\ 0)$

эцэст нь:

$succ\ 0$

үүнээс бид эцсийн утга болох 1-ийг олж авах бөгөөд энэ нь дараа нь v -тэй холбогдоно.

Lazy Evaluation

Нэрээр нь үнэлэхдээ дахин бичих явцад зарим нэг давхцал гарч болох тул нэг редексийг нэгээс олон удаа үнэлэх шаардлагатай байж болно. Бидний хэлэлцэж буй жишээн дээр D функцийн улмаас редекс

(succ 0) давхардсан бөгөөд D-ийн биеийг бүрдүүлдэг нөхцөлт илэрхийлэлд хоёр дахин⁴ буурсан байна. Энэ бол аргументын үнэлгээг функц хэрэгжсэний дараа хойшлуулснаас болж байгаа юм (мөн энэ нь нэрээр нь үнэлэх нь утга сарнихад утгыг барьж авах боломжийг олгодог). Гэхдээ энэ нь үр ашгийн хувьд их зардлатай байдаг (учир нь давхардсан редекс нь ихээхэн хэмжээний тооцоолол шаарддаг).

Энэ асуудлыг арилгах, нэрээр нь үнэлэх давуу талыг хадгалахын тулд, Lazy стратеги нь нэрээр ийм байдлаар үргэлжилдэг боловч анхны удаа "хуулбар"-тай учирсан тохиолдолд түүний утгыг хадгалдаг бөгөөд хэрэв ижил хувилбарын өөр хуулбар тааралдвал түүнийг ашиглах болно. Нэгэнт төлөв байхгүй тул ижил илэрхийллийн олон хуулбарыг үнэлэх нь үргэлж ижил утгатай дуусдаг тул lazy стратеги нь стратегийн ижил утгыг нэрээр нь тооцоолох баталгаа болдог.

Нэрийн болон lazy стратеги нь *call by need* стратегийн жишээ бөгөөд зөвхөн тооцоололд шаардлагатай болсон үед л редексийг бууруулдаг.

11.2.4 Comparison of the Strategies

Сүүлийн дэд хэсэгт нэрийн стратеги нь утгын стратеги сарних үед утгыг үүсгэдэг. Энэ хоёр стратеги нь ижил илэрхийлэлд ялгаатай утгыг бий болгож чадах уу гэсэн асуултыг тавих нь ухаалаг хэрэг юм.

Энэ асуултын хариултыг *цэвэр* функциональ парадигмын хамгийн чухал шинж чанаруудын нэгийг илэрхийлдэг дараах теоремоор өгсөн болно. Хэлний бүх хувьсагч нь зарим *fn*-ээр холбогдсон бол хэл дээрх илэрхийлэл *битүү* гэж бид хэлдэг. Тиймээс 11.2.1-ээс харахад *анхдагч утга* гэж функциональ утгыг оруулаагүй анхны төрлийн утгыг (бүхэл тоо, boolean, тэмдэгт гэх мэт) ойлгодог гэдгийг санацгаая.

Теорем 11.1 *exр-г битүү илэрхийлэл гэж үзье. Хэрэв exр нь 11.2.3-т заасан гурван стратегийн аль нэгийг ашиглан анхдагч утга val болж буурвал нэрийн стратегийг ашиглан exр нь val болгон бууруулна. Хэрэв exр нь нэрийн стратегийг ашиглан сарниж байгаа бол нөгөө хоёр стратегийн дагуу мөн адил хуваагдана.*

Теорем нь (битүү) илэрхийлэл нэг стратегид val гэсэн анхдагч утгыг гаргаж, өөр стратеги⁵ ашиглан өөр анхдагч утгыг val² гаргаж болохыг үгүйсгэж байгааг анхаарцгаая. Иймээс хоёр стратеги нь зөвхөн нэг нь утгыг тодорхойлж байхад нөгөө нь сарниж байдгаараа л ялгаатай байж болно, үүнийг бидний жишээн дээр харсан. Бид теоремыг хэрхэн нотлох талаар дэлгэрэнгүй тайлбарлаж чадахгүй ч дараахь үндсэн үзүүлэлт нь түүний хүчинтэй байдлын суурь гэдгийг онцлон тэмдэглэх нь зүйтэй.

⁴ Цэвэр байна гэдэг нь редексийн хуулбаруудын талаар ярьдаггүй, харин редекс (succ 0) нь хоёр үлдэгдэлд байрлаж, түс бүр нь нөгөөгөөсөө хамааралгүйгээр буурсан гэж хэлэх болно.

⁵ Утга нь анхдагч гэсэн таамаглалыг арилгавал теорем хүчингүй болно. Үнэлгээг хийсвэрлэлээр хийдэггүй гэж үзвэл үнэн хэрэгтээ хоёр стратеги нь ялгаатай (функциональ) утгыг гаргаж чадна. Жишээ болгон $I = \text{fn } x \Rightarrow x$ ба $P = \text{fn } x \Rightarrow (\text{fn } y \Rightarrow y \ x)$ гэж тодорхойл. $P \ (I \ I)$ нэр томъёо нь нэрийн стратегийн дагуу $\text{fn } y \Rightarrow y \ (I \ I)$ болж буурдаг бол нэмэлт утгын стратегийг ашигласнаар $\text{fn } y \Rightarrow y \ I$ болж буурдаг. Энэ ялгаа нь програмчлалын хэлэнд тийм ч чухал биш (бид үндсэн төрлийн утгыг тооцоолохыг хүсдэг). 11.6-д бид хийсвэрлэл байгаа тохиолдолд бууралт бас тохиолддог хийсвэр тооцоог тодорхойлох болно. Энэхүү тооцоолол нь нийлбэрийн шинж чанарыг (11.1 теоремоос илүү хүчтэй бөгөөд ерөнхий) хангаж байгаа бөгөөд үүнийг бидний хэсэг дээр авч үзэх болно. 11.6.

Аливаа стратегийг тухайн орчинд тохируулсан байхад нэг илэрхийллийн бүх тохиолдлын үнэлгээ нь үргэлж ижил утгыг өгдөг.

Гадаад нөлөө илэрвэл шууд хуурамчаар үйлчилдэг энэ үзүүлэлтийг олон зохиогчид цэвэр функциональ хэлний шалгуур болгон авдаг: хэл нь энэ нөхцлийг хангасан тохиолдолд цэвэр функциональ байдаг. Энэ бол 11.4-р хэсэгт буцаж ирэх функциональ програмын талаар *дүгнэлт* хийх боломжийг олгодог маш чухал үзүүлэлт юм. Зөвхөн энэ үзүүлэлтээрээ л lazy үнэлгээ зөв болохыг үргэлжлүүлэхээсээ өмнө дахин сануулъя. Илэрхийллийн үнэлгээгээр олж авсан утга нь ижил илэрхийллийн өөр хуулбартай тулгарах үед өөрчлөгдөхгүй (энэ хооронд орчин өөрчлөгдөөгүй нь тодорхой).

Өмнөх теоремын талаас авч үзвэл нэрийн стратеги бүх боломжит стратегиас хамгийн ерөнхий нь байдаг тул утгын стратегид ямар сонирхол байгааг асууж болно. Түүгээр ч зогсохгүй, үр ашгийн шалтгаан нь бид зөвхөн call-by-need стратегийг хэрэгжүүлэхийг санал болгож байгаа юм шиг санагдаж байна, учир нь ашиггүй редексийг яагаад бууруулах нь тодорхойгүй, улмаар тооцоолох цагийг дэмий үрдэг. Гол нь үр ашгийн асуудал тийм ч тодорхой биш байгаа юм. call-by-need стратегийг хэрэгжүүлэх нь ерөнхийдөө энгийн call-by-value стратегиас илүү өртөгтэй байдаг. Утгын стратеги нь заримдаа шаардлагагүй ажил хийдэг байсан ч ердийн архитектурт үр дүнтэй хэрэгжүүлэлттэй байдаг (Зураг 11.2 дахь K функцийн хоёр дахь аргументтай адил функцын их биед шаардлагагүй аргументыг үнэлэх бүрт). Гэсэн хэдий ч энэ сүүлийн тохиолдлыг хийсвэр үнэлгээний техникээр үр дүнтэйгээр шинжилж, ашиггүй аргуудыг тодорхойлохыг оролддог. Түүгээр ч зогсохгүй императив бүтэцтэй агуулсан функциональ хэл дээр утгаар бууруулах стратеги нь эдгээр императив үзүүлэлтүүдтэй илүү сайн нийцдэг.

Энэ бүлгийн эхэнд дурдсан функциональ хэлнүүдийн дотроос Lisp, Scheme, ML, Erlang зэрэг нь утгын стратегийг (мөн тэд үндсэн императив талуудыг багтаасан учраас) ашигладаг бол Миранда, Хаскелл (цэвэр функциональ хэлүүд) нар lazy үнэлгээг ашигладаг.

11.3 Programming in a Functional Language

Сүүлийн хэсэгт бидний тайлбарласан механизмууд нь тооцоолж болох функц бүрийн программыг илэрхийлэхэд хангалттай (тэдгээр нь Тьюринг-гүйцэд хэлийг бүрдүүлдэг). Энэ бол функциональ хэл бүрийн гол цөм нь хэдий ч үүнийг жинхэнэ програмчлалын хэл болгон ашиглахад хэтэрхий хатуу байдаг. Тиймээс функциональ хэл бүр энэ цөмийг өөр өөр янзын механизмуудтай илүү өргөн контекст дотор багтаасан байдаг бөгөөд энэ нь програмчлалыг хялбар, илүү илэрхийлэх чадвартай болгох зорилготой.

11.3.1 Локал орчин

Бидний өнөөг хүртэл ашигласан орчны глобал тодорхойлолтуудын механизм нь орчин үеийн хэлний хувьд хэтэрхий жижиг бүтэц. Уламжлалт хэлнүүдийн нэгэн адил хязгаарлагдмал хамрах хүрээтэй тодорхойлолтуудыг нэвтрүүлэх тодорхой механизмуудыг өгөх нь зүйтэй, жишээлбэл:

```
let x = exp in exp1 end
```

Энэ нь зөвхөн exp1-г багтаасан хамрах хүрээ дэх exp-ийн утгад x-г холбохыг танилцуулж байна.

Үнэнийг хэлэхэд, функциональ илэрхийлэл байгаа нь бодит параметруудтэй холбогдсон функцийн формал параметруудээс бүрдэх шаталсан хамрах хүрээ болон холбогдох орчныг аль хэдийн нэвтрүүлж байна. Үнэлгээний үүднээс авч үзвэл

```
let x = exp in exp1 end
```

бүтцийг дараах бүтцийн синтакс sugar гэж үзэж болно

```
(fn x => exp1) exp.
```

Локал хамрах хүрээг ашиглах нь бас чухал бөгөөд тэдэнд зориулсан тусгай синтаксийг нэвтрүүлэхийг зөвтгөдөг.

11.3.2 Interactiveness

Функциональ хэл бүр интерактив орчинтой байдаг. Хэл нь хийсвэр машины үнэлдэг илэрхийллүүдийг оруулан утгыг нь буцаана. Тодорхойлолт нь глобал орчныг өөрчилдөг тодорхой илэрхийллүүд (мөн утга буцааж болно).

Тодорхойлолтыг орчинд анх удаа хадгалахад компайлдсан код үүсгэдэг үр ашигтай хэрэгжүүлэлтүүд байдаг ч энэ загвар нь интерпретатор дээр суурилсан хэрэгжилтийг шууд санал болгодог.

11.3.3 Pattern Matching

Рекурсив функциональ програмчлалын хамгийн ядаргаатай талуудын нэг бол "if" гэсэн илэрхийллүүдийн тусламжтайгаар терминалын тохиолдлуудыг зохицуулах явдал юм. Жишээлбэл, Фибоначчийн цувааны n-р гишүүнийг буцаадаг (маш үр ашиггүй) функцийг авч үзье.

```
fun Fibo n = if n=0 then 1
             else if n=1 then 1
                   else Fibo(n-1)+Fibo(n-2);
```

ML болон Haskell гэх мэт зарим хэлэнд байдаг паттерн тааруулах (pattern matching) механизм нь бидэнд дараах тодорхойлолтыг (дээрхтэй тэнцүү) өгөх боломжийг олгодог.

```
fun Fibo 0 = 1
  | Fibo 1 = 1
  | Fibo n = Fibo(n-1)+Fibo(n-2);
```

"|" тэмдэгтийг "эсвэл" гэж уншина. Тодорхойлолтын салаа бүр нь функцийн өөр өөр тохиолдолтой байна. Энэ тодорхойлолтын хамгийн сонирхолтой хэсэг нь формал параметрууд юм. Тэдгээр нь ялгагч байхаа больсон ч *паттерн* байна, өөрөөр хэлбэл хувьсагч (сүүлийнх нь жишээ), тогтмол (бусад тохиолдолд) болон хэлний төрлийн системээс хамаарах бусад бүтцээс бүрдсэн илэрхийлэл байж болно (бид доор жагсаалтаар жишээг харуулна). Паттерн бол нэг янзын схем, бодит параметртэй тааруулах загвар юм. Функцийг бодит параметрт хэрэглэх үед паттернтай харьцуулж (программд гарч ирэх дэс дарааг ашиглан) бодит параметртэй тохирох эхний паттернд харгалзах их биеийг сонгоно.

Паттерн тааруулах механизм нь ялангуяа бүтэцлэгдсэн төрлүүд, жишээ нь жагсаалтыг ашиглах үед уян хатан байдаг.⁶ ML синтакс дээр жагсаалтыг дөрвөлжин хаалтанд, элементүүдийг таслалаар тусгаарласан тэмдэглэнэ. Жишээлбэл

```
["one", "two", "three"]
```

Энэ нь гурван тэмдэгт мөрийн мөрийн жагсаалт. Оператор :: нь "cons" гэсэн утгатай (энэ нь жагсаалтын толгойд элемент нэмэх оператор юм):

```
"zero>::["one", "two", "three"]
```

Энэ нь жагсаалтын утга болох илэрхийлэл юм:

```
["zero", "one", "two", "three"].
```

Эцэст нь nil бол хоосон жагсаалт юм. Тиймээс

```
"four>::nil
```

гэвэл ["дөрөв"] утгатай байна. Паттерн тохируулгыг ашиглан бид нийтлэг жагсаалтын уртыг тооцоолох функцийг тодорхойлж болно, тухайлбал:

```
fun length nil = 0
  | length e::rest = 1 + length(rest);
```

Паттернд ашигласан нэрийг бодит параметрийн хэсгийг заах формал параметр болгон ашигладаг болохыг анхаарна уу. Ердийн тодорхойлолттой харьцуулахад товч тодорхой байх давуу тал нь илэрхий байх ёстой:

```
fun length list = if list = nil then 0
                  else 1 + length(tl list);
```

Үүнээс гадна энэ тодорхойлолт нь толгой элементийг арилган гаргаж авсан жагсаалтыг байгуулахын тулд *сонгох (selection)* функцийг (бид үүнийг tl гэж бичсэн) нэвтрүүлэхийг шаарддаг. Нөгөө талаас сонголт хийх үйлдэл нь паттерн тааруулахад далд явагддаг.

Нэг чухал шаардлага нь хувьсагч нэг паттернд хоёр удаа гарах боломжгүй. Жишээлбэл, жагсаалтад хэрэглэсэн функцийн дараах "тодорхойлолт"-ыг авч үзье, энэ нь зөвхөн жагсаалтын эхний хоёр элемент тэнцүү байвал true буцаана:

```
fun Eq nil = false
  | Eq [e] = false
  | Eq x::x::rest = true
  | Eq x::y::rest = false ;
```

Гурав дахь паттерн нь эхний хоёр элементийн тэнцүүг шалгахад хэрэглэгддэг х хувьсагчийг хоёр удаа агуулж байгаа тул энэ тодорхойлолт нь синтаксийн хувьд буруу. Паттерн тааруулах механизм нь бодит параметрийн *хэлбэрийг* паттернтай тохирч байгаа эсэхийг шалгах нэг арга бөгөөд түүнд

⁶ Функциональ талбарт жагсаалт нь *хувиршгүй (immutable)* өгөгдөл байдаг тул жишээлбэл, Python-д үүнийг tuple гэж нэрлэх болно гэдгийг санаарай.

агуулагдах утгууд нь тодорхой харилцаатай байгаа эсэхийг шалгадаггүй. Өөрөөр хэлбэл, паттерн тааруулах нь нэгдмэл биш, илүү ерөнхий механизм (мөн хэрэгжүүлэхэд илүү төвөгтэй) бөгөөд бид үүнийг логик програмчлалын парадигмын хүрээнд авч үзэх болно (Бүлэг 12).⁷

11.3.4 Функционал хэв маягаар программчлах

Функциональ хэв маягаар програмчлах нь (жижиг) функцийн (том) багцаар удирддаг *хувиршгүй өгөгдлийн* эргэн тойронд байдаг. Өндөр эрэмбийн функц ашигласнаар ерөнхий програмчлалын схемийг функц гэж тодорхойлж болох бөгөөд улмаар програмын тусгай ажиллагааг байгуулахын тулд тэдгээрийг үүсгэж болно. Энгийн жишээ бол "*map, filter, fold pattern*" гэж нэрлэгддэг бөгөөд энэ нь императив хэв маяг дахь "дарааллын шугаман скан" -ын функциональ эквивалент юм.

`map(f,list_of_data)` функц нь (ихэвчлэн урьдчилан тодорхойлсон) `list_of_data`-ын дурын элементэд `f`-г хэрэглэн гаргаж авсан жагсаалтыг буцаах функц юм.

Дараах императив Python кодын оронд бид зүгээр л `res=map(f,өгөгдлийн_жагсаалт)` гэж бичнэ:

```
res=[]
for e in list_of_data:
    res.append(f(e))
```

Функц `fold(f,init,list_of_data)` нь (ихэвчлэн урьдчилан тодорхойлсон) функц бөгөөд `f` хоёртын функцийг авч, `list_of_data` бүх элементүүдэд хэрэглэж, хуримтлагдсан үр дүнг буцаана; `init` нь `fold` эхлэхэд хэрэглэгддэг анхны утга юм. Жишээлбэл, `fold(fn x,y=>x+y, 0, list_of_int)` нь `list_of_int`-ийн бүх элементүүдийн нийлбэрийг буцаана.⁸

`filter(p,list_of_data)` функц нь `list_of_data`-ын `p` предикат (нөхцөл)-ийг хангасан эдгээр элементүүдийг цуглуулан байгуулсан жагсаалтыг буцаадаг (ихэвчлэн урьдчилан тодорхойлсон) функц.

Өндөр эрэмбийн бусад багц функцууд нь өөр олон програм схемийг товч (болон үр ашигтай) илэрхийлэх боломжийг олгодог. Бидний хэлэхийг хүсч буй хамгийн чухал зүйл бол програм схемийг өргөнөөр ашиглах нь кодын модулчлалыг нэмэгдүүлдэг. Программ нь илүү модульчлагдсан байх тусам бид тухайн програмын тестчилэлийг (эсвэл баталгаажуулалт, эсвэл зөв эсэхийг нотлох) илүү хялбараар тооцож болно.

Бусад програмчлалын парадигмуудыг ашиглах боломжийг олгодог програмчлалын хэлийг ашиглан *функциональ програмчлалын хэв маягийг* ашиглаж болох нь ойлгомжтой. Хэл нь өндөр эрэмбийн функцуудээр хангасны дараа төлөвт суурилсан тооцоолол хийхээс зайлсхийж том программ бичихэд хялбар болно. Энэ нь жишээлбэл, том "функциональ дэд олонлогтой" бөгөөд шаардлагатай бүх, урьдчилан тодорхойлсон өндөр эрэмбийн функцуудыг агуулсан Scala тохиолдол

⁷ Паттерн тааруулах хэрэгжилт нь `ifs`-ийн дараалал руу "илэрхий" хөрвүүлэлт хийхээс өөр зүйл биш юм. `Eq`-ийн хувьд гурав дахь паттернд *дурын төрлийн* хоёр элементийн тэнцүүг шалгах шаардлагатай болно. Хэрэв бид хэлээр тэнцүүг тодорхойлдоггүй утгуудтай бол (жишээлбэл, функциональ утгын хувьд тохиолддог) тэгш байдлын тестийг хэрэгжүүлэх боломжгүй болно.

⁸ Нэргүй функцийг (ламбда) `fold` аргумент болгон ашиглахыг харна уу.

юм.⁹ Түүнчлэн, Scala-д нэрийг хоёр өөр бүтцээр танилцуулж болно: `var` нь дахин оноож болох нэрийг зарладаг бол `val` нь нэрийг ямар ч засвар оруулах байхаар утгад холбодог. `val` ба зөвхөн засварлах боломжгүй утгуудыг ашигласнаар бид аливаа гадаад нөлөөнөөс зайлсхийж, жинхэнэ функциональ програмчлалын хэв маягийг бий болгоно. Цэвэр програмчлалын хэлний онцлог нь ямар ч гадаад нөлөө¹⁰ байхгүй байх нь программын зөв байдлыг хянахад хялбар болгодог (11.4-р хэсгийг үзнэ үү).

11.3.5 Types

11.3.6 Infinite Objects

11.3.7 Imperative Aspects

11.4 An Assessment

11.5 Implementation: The SECD Machine

11.6 Fundamentals: The λ -Calculus

⁹ Мөн Python-ийн функцууд нь нэгдүгээр зэрэглэлийнх. Гэсэн хэдий ч Scala-аас ялгаатай нь Python нь нэрийг дахин хуваарилах боломжийг хязгаарлах хэл шинжлэлийн бүтэцгүй байдаг. Тиймээс цэвэр функциональ дэд бүлгийг тусгаарлах нь илүү асуудалтай байдаг.

¹⁰ Эсвэл Erlang, Scala шиг эдгээр хэлний хязгаарлагдмал, хязгаарлагдмал гадаад нөлөө нь цэвэр биш боловч аливаа гадаад нөлөөг тусгай функцүүдэд инкапсуляц хийхийг шаарддаг.